

PATCH_GUIDE.md — Alux Plaza (jinarous)

Read this first. This guide maps every fix to a file path, gives verification commands, and includes a rollout checklist.

Table of Contents

- 1. Architecture Overview
- 2. File-Path Map by Domain
 - Auth & Session Management
 - Rate Limiting & Throttling
 - CSRF Protection
 - Shield / WAF
 - Database & Migrations
 - Frontend Security
 - Config & Environment
 - Monitoring & Alerting
 - Deployment
- 3. Verification Commands
- 4. Rollout Checklist
- 5. Rollback Procedures

Architecture Overview

Layer	Stack	Key Files
Backend	Node.js 20+ / Express (ESM)	backend/src/index.js, backend/src/config.js
Database	PostgreSQL 16	backend/migrations/*.sql, backend/src/db.js
Auth	JWT (double cookie)	backend/src/middleware/auth.js, backend/src/middleware/clientAuth.js
Rate Limit	Postgres-backed (express-rate-limit)	backend/src/middleware/rate-limit.js, backend/src/lib/rate-limit-store.js
WAF	Custom Shield (pattern + volume)	backend/src/middleware/shieldMiddleware.js, backend/src/shield/*.js
Frontend	React 19 / Vite / TypeScript / Tailwind	frontend/src/lib/api.ts, frontend/src/lib/security.ts
Deploy	Render (backend) + Vercel (frontend)	render.yaml, docker-compose.yml

File-Path Map by Domain

Auth & Session Management

Fix	File Path	Notes
Admin JWT verification + blocklist	backend/src/middleware/auth.js	Checks adminToken cookie or Authorization header; rejects role: 'client' tokens; queries token_blocklist table
Client JWT verification + blocklist	backend/src/middleware/clientAuth.js	Mirrors auth.js but requires role: 'client'; uses same token_blocklist table
Admin login (bcrypt, lockout, dummy hash)	backend/src/routes/admin.js	DUMMY_HASH prevents timing attacks on email enumeration; per-account lockout via failed_login_count / locked_until
Client login	backend/src/routes/clientAuth.js	Same lockout pattern as admin
Token blocklist persistence	backend/migrations/005_add_token_blocklist_and_rate_limits.sql	token_blocklist(jti, expires_at) —shared across workers
Admin lockout schema	backend/migrations/017_add_admin_lockout.sql	failed_login_count, locked_until on admin_users
Client lockout schema	backend/migrations/015_add_client_lockout.sql	Same fields on clients table
Logout / revocation	backend/src/routes/admin.js (POST /logout)	Writes JTI to token_blocklist with expiry
Boot-time admin bootstrap	backend/src/index.js	ADMIN_BOOTSTRAP_EMAIL + ADMIN_BOOTSTRAP_PASSWORD; refuses to boot in production if still set after admins exist

Rate Limiting & Throttling

Fix	File Path	Notes
General API limiter (100/15min)	backend/src/middleware/rate-limit.js	limiter —uses PostgresRateLimitStore
Auth endpoint limiter (10/15min)	backend/src/middleware/rate-limit.js	authLimiter —key prefix auth:
Contact form limiter	backend/src/middleware/rate-limit.js	contactLimiter —env-driven CONTACT_RATE_LIMIT_*
Postgres store implementation	backend/src/lib/rate-limit-store.js	PostgresRateLimitStore class; rate_limits table (migration 005)
Rate limits schema	backend/migrations/005_add_token_blocklist_and_rate_limits.sql	rate_limits(key, count, reset_time)

CSRF Protection

Fix	File Path	Notes
Double-submit cookie (set)	backend/src/middleware/csrf.js	setCsrfCookie —httpOnly: false, SameSite=strict, 24h expiry
Double-submit cookie (verify)	backend/src/middleware/csrf.js	verifyCsrfToken —skips GET/HEAD/OPTIONS; uses timingSafeEqual
Frontend CSRF fetch wrapper	frontend/src/lib/security.ts	secureFetch() reads csrfToken cookie, sets x-csrf-token header
Alternate CSRF fetch wrapper	frontend/src/lib/secureFetch.ts	Same pattern; consolidate into one eventually
CSRF cookie registration	backend/src/index.js	app.use(setCsrfCookie) before routes; app.use(verifyCsrfToken) after body parsing

Shield / WAF

Fix	File Path	Notes
Main Shield middleware	backend/src/middleware/shieldMiddleware.js	Runs isBlocked → recordRequest → scanRequest; exempts /api/contact from signature scan
IP blocklist (DB-backed)	backend/src/shield/blocklist.js	blocked_ips table (migration 011); upsert per IP; severity-based durations
Brute-force / volume guard	backend/src/shield/bruteForceGuard.js	In-memory failedLogins and requestCounts Maps; not cluster-safe
Attack detector	backend/src/shield/detector.js	Normalization pipeline (URL-decode ×3, HTML entities, NFKC, lowercasing, comment stripping) + regex patterns for SQLi, XSS, path traversal
Risk score engine	backend/src/shield/riskScore.js	Computes 0-100 score from client_compliance_status; per-framework breakdown
Event logger	backend/src/shield/eventLogger.js	Writes to security_events table
Blocked IPs schema	backend/migrations/011_create_shield_tables.sql	blocked_ips, security_events

Table 5 – continued

Fix	File Path	Notes
Compliance tracker schema	backend/migrations/ 012_create_compliance_tracker.sql	compliance_items, client_compliance_status

Database & Migrations

Fix	File Path	Notes
Connection pool	backend/src/db.js	pg.Pool; SSL in production; max: 20; retry logic with exponential backoff
Migration runner	backend/src/migrations-runner.js	Applies .sql files in filename order
Boot-time migration trigger	backend/src/db.js	runMigrations(pool) called inside initDb()
All schema files	backend/migrations/001_init.sql – 018_seed_compliance_frameworks.sql	Applied automatically on boot; gap at 008 is harmless
Metrics persistence	backend/migrations/002_add_metrics_table.sql	metrics(key, value) for cumulative stats
Audit logs schema	backend/migrations/007_add_audit_logs.sql	audit_logs table
Contact indexes	backend/migrations/ 009_add_contact_indexes.sql	Indexes on contacts(status, created_at) etc.
Risk score shares	backend/migrations/ 013_create_risk_score_shares.sql	risk_score_shares table

Frontend Security

Fix	File Path	Notes
API base URL	frontend/src/lib/api.ts	VITE_API_BASE_URL; runtime warning if missing in production
Build-time env check	frontend/scripts/check-env.js	prebuild hook; fails build if VITE_API_BASE_URL unset
Secure fetch (CSRF + credentials)	frontend/src/lib/security.ts	secureFetch() attaches CSRF token, credentials: 'include'
Honeypot check	frontend/src/lib/security.ts	isHoneypotTriggered() checks #website_url field
CSP nonce generation	backend/src/middleware/helmetConfig.js	attachCspNonce –16-byte random per request
Strict Helmet headers	backend/src/middleware/helmetConfig.js	CSP with nonces, HSTS (1yr, preload), frame-ancestors 'none'

Config & Environment

Fix	File Path	Notes
Boot-time config validation	backend/src/config.js	Refuses to start if JWT_SECRET missing, <32 chars, low entropy, or weak pattern; enforces CORS_ORIGIN in production
Backend env template	backend/.env.example	DATABASE_URL, JWT_SECRET, CORS_ORIGIN, ADMIN_EMAIL, SENTRY_DSN, ALERT_WEBHOOK_URL
Frontend env template	frontend/.env.example	VITE_API_BASE_URL
Root env template	env.example	Docker Compose variables

Monitoring & Alerting

Fix	File Path	Notes
Sentry integration	backend/src/monitoring.js	SENTRY_DSN optional; captureError logs + sends to Sentry
Alert webhook (Slack/Discord)	backend/src/monitoring.js	ALERT_WEBHOOK_URL; 5-min throttle per key; max 500 entries
Request stats	backend/src/stats.js	In-memory latencies + delta-persisted counters; persistStats() every 10s
Status endpoint	backend/src/routes/status.js	Returns aggregated DB totals (cluster-safe)
Deep health check	backend/src/index.js	GET /api/health/deep —queries SELECT 1
Pino HTTP logging	backend/src/index.js	pinoHttp; ignores /api/health

Deployment

Fix	File Path	Notes
Render blueprint	render.yaml	Free tier; CLUSTER_MODE=false; auto-migrate; health check / api/health
Docker Compose (dev)	docker-compose.yml	Postgres 16, backend, optional frontend; no Redis
Docker Compose (prod)	docker-compose.prod.yml	Slimmer variant
Backend Dockerfile	backend/Dockerfile	Node 20 Alpine
Frontend Dockerfile	frontend/Dockerfile	Nginx serving static build
Frontend nginx config	frontend/nginx.conf	SPA fallback, gzip, security headers
Vercel config	frontend/vercel.json	SPA rewrite rules

Verification Commands

1. Backend Boot & Config

```
cd backend
npm ci

# Verify config validation (should refuse to start without JWT_SECRET)
unset JWT_SECRET
node src/config.js 2>&1 | grep -q "Refusing to start" && echo "PASS: config blocks missing JWT_SECRET"
↪ || echo "FAIL"

# Verify with valid config
export JWT_SECRET=$(node -e "console.log(require('crypto').randomBytes(48).toString('hex'))")
export DATABASE_URL="postgres://appuser:password@localhost:5432/appdb"
export CORS_ORIGIN="http://localhost:3000"
node src/config.js && echo "PASS: config accepts valid env" || echo "FAIL"
```

2. Database & Migrations

```
cd backend

# Verify migration order (gap at 008 is expected)
ls migrations/*.sql | sort

# Verify migrations apply cleanly
npm run migrate

# Verify all expected tables exist
psql "$DATABASE_URL" -c "\dt" | grep -E "admin_users|clients|contacts|token_blocklist|rate_limits|
↪ blocked_ips|security_events|metrics|audit_logs|compliance_items|client_compliance_status|
↪ risk_score_shares"
```

3. Auth & Session

```
# Start backend
npm run dev &
BACKEND_PID=$!
sleep 3

# Health check
curl -s http://localhost:4000/api/health | jq -e '.status == "ok"' && echo "PASS: health" || echo "FAIL"

# Deep health
curl -s http://localhost:4000/api/health/deep | jq -e '.database == "connected"' && echo "PASS: deep
↪ health" || echo "FAIL"

# Admin login without CSRF should fail
```

```

curl -s -X POST http://localhost:4000/api/admin/login -H "Content-Type: application/json" -d
↪ '{"email":"a@b.com","password":"x"}' | grep -q "CSRF token missing" && echo "PASS: CSRF enforced"
↪ || echo "FAIL"

# Admin login with CSRF should proceed (and fail credentials, not CSRF)
CSRF=$(curl -s -c - http://localhost:4000/api/health | grep csrfToken | awk '{print $7}')
curl -s -X POST http://localhost:4000/api/admin/login -b "csrfToken=$CSRF" -H "x-csrf-token: $CSRF"
↪ -H "Content-Type: application/json" -d '{"email":"a@b.com","password":"x"}' | grep -q "Invalid
↪ credentials" && echo "PASS: auth flow works" || echo "FAIL"

kill $BACKEND_PID

```

4. Rate Limiting

```

# Should return 429 after 10 login attempts in 15 minutes
for i in {1..12}; do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:4000/api/admin/login -b
  ↪ "csrfToken=$CSRF" -H "x-csrf-token: $CSRF" -H "Content-Type: application/json" -d
  ↪ '{"email":"a@b.com","password":"x"}'
done | tail -1 | grep -q "429" && echo "PASS: auth rate limit triggers" || echo "FAIL"

```

5. Shield / WAF

```

# SQLi should be blocked
curl -s -X POST http://localhost:4000/api/contact -b "csrfToken=$CSRF" -H "x-csrf-token: $CSRF" -H
↪ "Content-Type: application/json" -d '{"name":"x","email":"a@b.com","message":"1 UNION SELECT *
↪ FROM admin_users"}' | grep -q "Request blocked" && echo "PASS: Shield blocks SQLi" || echo "FAIL"

# Path traversal should be blocked
curl -s "http://localhost:4000/api/health?file=../../../../etc/passwd" | grep -q "Request blocked" && echo
↪ "PASS: Shield blocks path traversal" || echo "FAIL"

# Contact form (exempt from signature scan) should NOT be blocked for benign text
curl -s -X POST http://localhost:4000/api/contact -b "csrfToken=$CSRF" -H "x-csrf-token: $CSRF" -H
↪ "Content-Type: application/json" -d '{"name":"x","email":"a@b.com","message":"We need to select a
↪ union representative"}' | grep -q "Request blocked" && echo "FAIL: false positive on contact" ||
↪ echo "PASS: contact exempt works"

```

6. Frontend Build

```

cd frontend
npm ci

# Should fail without VITE_API_BASE_URL
unset VITE_API_BASE_URL
npm run build 2>&1 | grep -q "VITE_API_BASE_URL is required" && echo "PASS: build fails without API
↪ base" || echo "FAIL"

# Should succeed with it
VITE_API_BASE_URL=http://localhost:4000 npm run build && echo "PASS: build succeeds" || echo "FAIL"

```

7. Docker Compose

```
# Validate compose files
docker compose config > /dev/null && echo "PASS: docker-compose.yml valid" || echo "FAIL"
docker compose -f docker-compose.prod.yml config > /dev/null && echo "PASS: docker-compose.prod.yml
↪ valid" || echo "FAIL"

# Full stack smoke test
docker compose up --build -d
sleep 10
curl -s http://localhost:3001/api/health | jq -e '.status == "ok"' && echo "PASS: Docker backend
↪ healthy" || echo "FAIL"
docker compose down
```

8. CI / Tests

```
cd backend
npm test
# Expected: all vitest suites pass (uses disposable Postgres)
```

Rollout Checklist

Pre-Deploy

- JWT_SECRET is set, 32 chars, entropy 3.5 bits/char, no dictionary words
- DATABASE_URL includes ?sslmode=require for managed Postgres (Neon, etc.)
- CORS_ORIGIN matches deployed frontend URL exactly (including https://)
- ADMIN_BOOTSTRAP_EMAIL and ADMIN_BOOTSTRAP_PASSWORD are set **only for first deploy**; remove after first login
- VITE_API_BASE_URL is set in frontend build environment
- All migrations run cleanly: `npm run migrate`
- `npm test` passes in backend/
- `npm run build` passes in frontend/
- `docker compose config` validates without errors
- CLUSTER_MODE is false for single-instance deployments (free tier)

Deploy

- Backend deployed and `/api/health` returns `{"status":"ok"}`
- `/api/health/deep` returns `{"database":"connected"}`
- Frontend loads without console warnings about missing VITE_API_BASE_URL
- Admin login page accessible; CSRF cookie (csrfToken) present
- Failed admin login returns 401 (not 500)
- 11th failed login from same IP returns 429
- Contact form submission succeeds with benign text
- Contact form with SQLi payload returns 403 from Shield
- Admin dashboard loads after login; adminToken cookie is httpOnly; Secure; SameSite=Strict
- Logout clears adminToken and revokes JTI in token_blocklist

Post-Deploy

- Remove ADMIN_BOOTSTRAP_EMAIL and ADMIN_BOOTSTRAP_PASSWORD from host env vars
 - Verify SENTRY_DSN is set in production (optional but recommended)
 - Verify ALERT_WEBHOOK_URL is set (optional)
 - Spot-check metrics table has non-zero requestCount
 - Check security_events table for any unexpected blocks (false positives)
 - Confirm blocked_ips has reasonable entries; no legitimate IPs permanently blocked
-

Rollback Procedures

Database Migration Rollback

Migrations are **forward-only** in this project. If a migration is bad:

1. Do **not** delete the migration file.
2. Write a **corrective migration** with the next sequential number.
3. The corrective migration must be idempotent (IF EXISTS / ON CONFLICT / CREATE OR REPLACE).

```
# Example: undoing a bad column addition
cat > backend/migrations/019_fix_bad_column.sql << 'EOF'
ALTER TABLE some_table DROP COLUMN IF EXISTS bad_column;
EOF
npm run migrate
```

Config / Env Rollback

If a bad env var is deployed:

1. Revert the env var in Render dashboard / Vercel dashboard / .env.
2. Trigger a redeploy (Render auto-deploys on env change; Vercel needs a new build).
3. Verify /api/health/deep passes.

Code Rollback

1. Revert the commit in Git.
2. Push to main.
3. Render auto-deploys; Vercel rebuilds on push.
4. Run the Verification Commands suite.

Emergency: Disable Shield

If Shield is blocking legitimate traffic and you need immediate relief:

```
# Temporarily exempt the affected path
# Edit backend/src/middleware/shieldMiddleware.js
# Add the path to SIGNATURE_SCAN_EXEMPT_PATHS, redeploy.
```

Or, to completely bypass Shield (not recommended except in emergencies):

```
# Comment out in backend/src/index.js:
# app.use(shield);
```

Always follow up by:

1. Checking `security_events` for the false-positive pattern.
2. Adjusting `detector.js` normalization or regex.
3. Re-enabling Shield.

Generated from codebase analysis. Last updated: 2026-08-23.